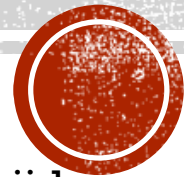


VENDING MACHINE CONTROLLER

Team 1

Younsuk Choi, Yuming (Leano) Wang, Muhammad Emaan Sajid

2025.06.24

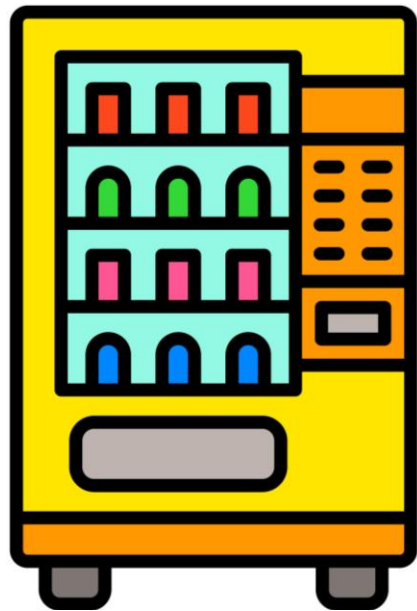


REQUIREMENTS AND FUNCTIONALITIES

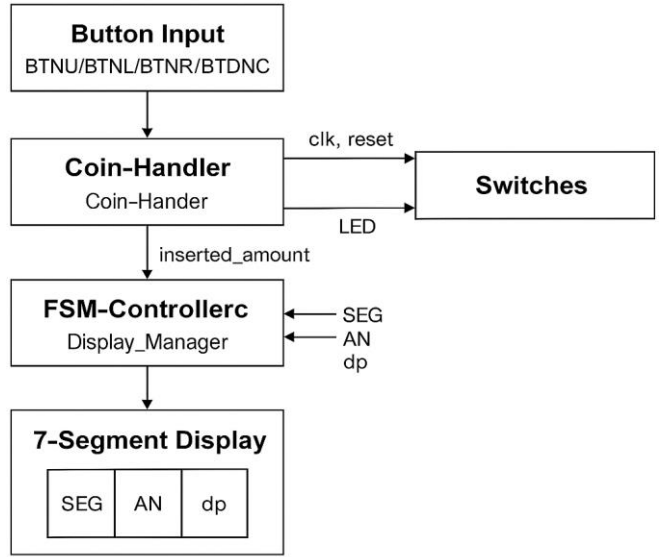
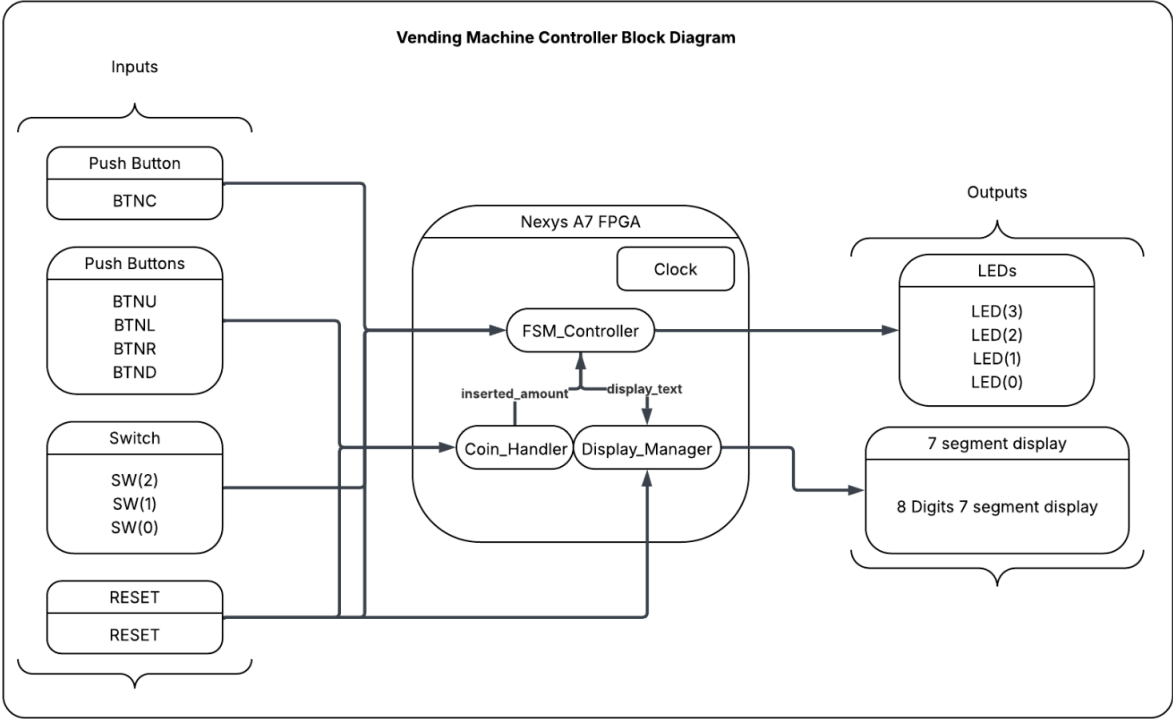
We started our project by formulating what functionalities we would like to achieve with this project. The goal of the project was to design and implement a fully functional vending machine controller using an FPGA, with a modular VHDL architecture, then later on design a custom PCB prototype.

The key functionalities/requirements of the vending machine was the following.

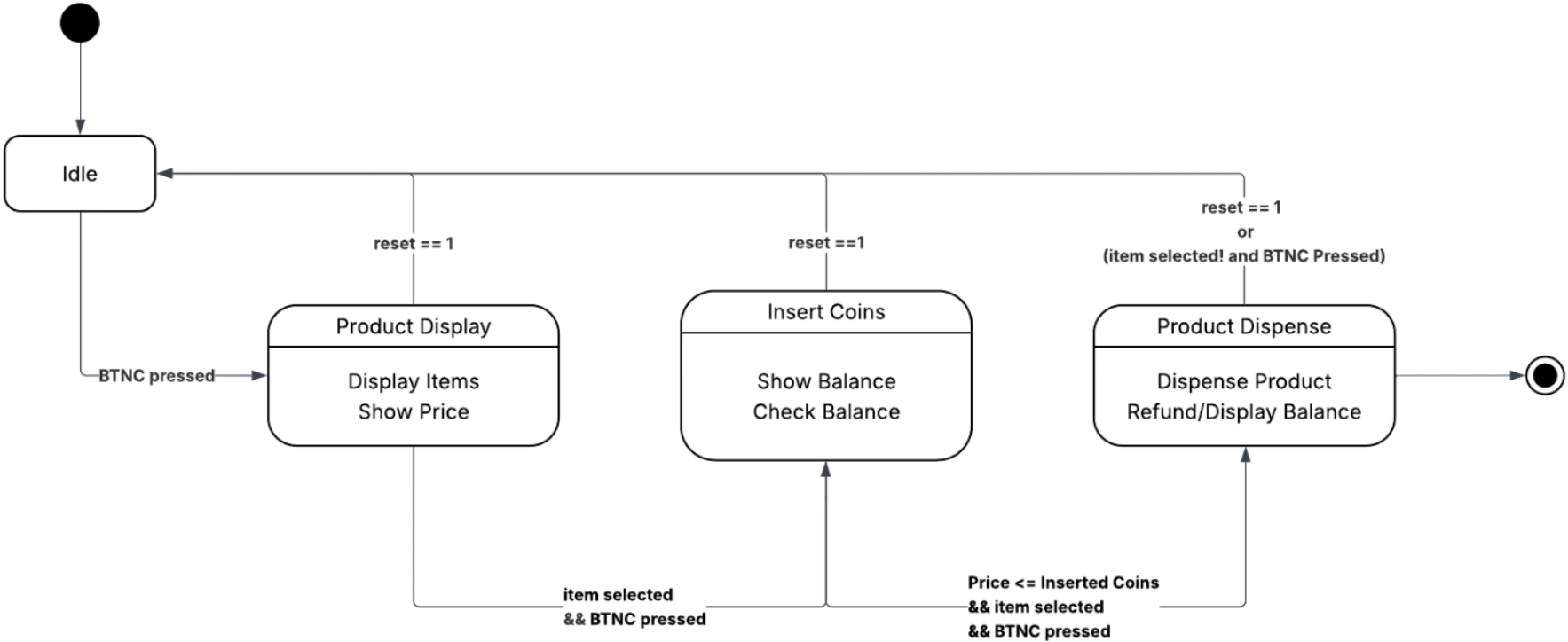
- Selection of items using switches
- Display the names and the prices of the items using 8 digits of seven segment display
- Insertion coins using push buttons
- Display the inserted coin amounts using 8 digits of seven segment display
- Display the current state using LEDs



CONCEPTS WITH DIAGRAMS_BLOCK DIAGRAM



CONCEPTS WITH DIAGRAMS STATE MACHINE



VHDL CODE IMPLEMENTATIONS

VENDINGMACHINE_TOP.VHD

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity VendingMachine_Top is
6  Port (
7      clk      : in  std_logic;
8      reset    : in  std_logic;
9      BTNC     : in  std_logic;
10     BTNU     : in  std_logic;
11     BTNL     : in  std_logic;
12     BTNR     : in  std_logic;
13     BTND     : in  std_logic;
14     SW       : in  std_logic_vector(2 downto 0);
15     LED      : out std_logic_vector(3 downto 0);
16     SEG      : out std_logic_vector(6 downto 0);
17     AN       : out std_logic_vector(7 downto 0);
18     dp       : out std_logic;
19 );
20 end VendingMachine_Top;
21
22 architecture Behavioral of VendingMachine_Top is
23     signal display_text_sig : std_logic_vector(63 downto 0);
24     signal inserted_amount_sig : unsigned(13 downto 0);
25 
```

```

26 begin
27     fsm_inst : entity work.FSM_Controller
28     port map (
29         clk      => clk,
30         reset    => reset,
31         BTNC     => BTNC,
32         SW       => SW,
33         inserted_amount => inserted_amount_sig,
34         LED      => LED,
35         display_text  => display_text_sig
36     );
37
38     coin_inst : entity work.Coin_Handler
39     port map (
40         clk      => clk,
41         reset    => reset,
42         BTNU     => BTNU,
43         BTNL     => BTNL,
44         BTNR     => BTNR,
45         BIND     => BIND,
46         inserted_amount => inserted_amount_sig(9 downto 0)
47     );
48
49     disp_inst : entity work.Display_Manager
50     port map (
51         clk      => clk,
52         reset    => reset,
53         display_text => display_text_sig,
54         SEG      => SEG,
55         AN       => AN,
56         dp       => dp
57     );
58
59 end Behavioral;
60 
```



VHDL CODE IMPLEMENTATIONS

FSM_CONTROLLER.VHD

PART 1

string_to_ascii

converts 8-characters

string to a 64-bit

std_logic_vector

format_price_to_string

converts 14-bit

unsigned integer(cents)

to 4-digit ASCII

character string

```

5 entity FSM_Controller is
6   Port (
7     clk       : in  std_logic;
8     reset     : in  std_logic;
9     BTNC      : in  std_logic;
10    SW        : in  std_logic_vector(2 downto 0);
11    inserted_amount : in  unsigned(13 downto 0);
12    LED       : out std_logic_vector(3 downto 0);
13    display_text  : out std_logic_vector(63 downto 0)
14  );
15 end FSM_Controller;
16
17 architecture Behavioral of FSM_Controller is
18
19   function string_to_ascii(s : string) return std_logic_vector is
20     variable result : std_logic_vector(63 downto 0) := (others => '0');
21   begin
22     for i in 0 to 7 loop
23       if i < s'length then
24         result(63 - i*8 downto 56 - i*8) := std_logic_vector(to_unsigned(character'pos(s(i+1)), 8));
25       else
26         result(63 - i*8 downto 56 - i*8) := x"20"; -- space
27       end if;
28     end loop;
29     return result;
30   end function;
31
32   function format_price_to_string(value : unsigned(13 downto 0)) return string is
33     variable str : string(1 to 4);
34     variable int_val : integer := to_integer(value);
35   begin
36     str(1) := character'val(int_val / 1000 mod 10 + character'pos('0'));
37     str(2) := character'val(int_val / 100 mod 10 + character'pos('0'));
38     str(3) := character'val(int_val / 10 mod 10 + character'pos('0'));
39     str(4) := character'val(int_val mod 10 + character'pos('0'));
40     return str;
41   end function;
42
43   type state_type is (IDLE, DISPLAY, INSERT, DISPENSE);
44   signal current_state, next_state : state_type;
45   signal BTNC_prev : std_logic := '0';
46   signal BTNC_edge : std_logic;
47   signal display_text_internal : std_logic_vector(63 downto 0);
48   signal selected_price : unsigned(13 downto 0) := (others => '0');
49

```



VHDL CODE IMPLEMENTATIONS

FSM_CONTROLLER.VHD

PART 2_PROCESSES

```

50 begin
51
52     display_text <= display_text_internal;
53
54     -- Detect rising edge of BTNC
55     process(clk)
56     begin
57         if rising_edge(clk) then
58             BTNC_prev <= BTNC;
59         end if;
60     end process;
61
62     BTNC_edge <= '1' when (BTNC = '1' and BTNC_prev = '0') else '0';
63
64     -- State register
65     process(clk, reset)
66     begin
67         if reset = '1' then
68             current_state <= IDLE;
69         elsif rising_edge(clk) then
70             current_state <= next_state;
71         end if;
72     end process;
73

```

```

74
75     -- FSM transitions
76     process(current_state, BTNC_edge, SW, inserted_amount, selected_price)
77     begin
78         case current_state is
79             when IDLE =>
80                 if BTNC_edge = '1' then
81                     next_state <= DISPLAY;
82                 else
83                     next_state <= IDLE;
84                 end if;
85
86             when DISPLAY =>
87                 if BTNC_edge = '1' and (SW = "001" or SW = "010" or SW = "100") then
88                     next_state <= INSERT;
89                 else
90                     next_state <= DISPLAY;
91                 end if;
92
93             when INSERT =>
94                 if BTNC_edge = '1' and inserted_amount >= selected_price then
95                     next_state <= DISPENSE;
96                 else
97                     next_state <= INSERT;
98                 end if;
99
100             when DISPENSE =>
101                 if BTNC_edge = '1' and SW = "000" then
102                     next_state <= IDLE;
103                 else
104                     next_state <= DISPENSE;
105                 end if;
106
107             when others =>
108                 next_state <= IDLE;
109         end case;
110     end process;
111
112     -- LED output
113     with current_state select
114         LED <= "0001" when IDLE,
115              "0010" when DISPLAY,
116              "0100" when INSERT,
117              "1000" when DISPENSE;

```

Moore's FSM



VHDL CODE IMPLEMENTATIONS

FSM CONTROLLER.VHD

```

118 -- Display logic
119 process(current_state, SW, inserted_amount, selected_price)
120     variable price_str      : string(1 to 4);
121     variable inserted_str   : string(1 to 4);
122     variable full_str       : string(1 to 8);
123     variable temp_change    : unsigned(13 downto 0);
124     variable chg_str        : string(1 to 4);
125 begin
126     case current_state is
127
128         when IDLE =>
129             if SW = "001" then
130                 selected_price <= to_unsigned(130, 14);
131                 display_text_internal <= string_to_ascii("01300000");
132             elsif SW = "010" then
133                 selected_price <= to_unsigned(150, 14);
134                 display_text_internal <= string_to_ascii("01500000");
135             elsif SW = "100" then
136                 selected_price <= to_unsigned(90, 14);
137                 display_text_internal <= string_to_ascii("00900000");
138             else
139                 selected_price <= (others => '0');
140                 display_text_internal <= string_to_ascii("      ");
141             end if;
142
143         when DISPLAY =>
144             case SW is
145                 when "001" =>
146                     selected_price <= to_unsigned(130, 14);
147                     display_text_internal <= string_to_ascii("colA0130");
148                 when "010" =>
149                     selected_price <= to_unsigned(150, 14);
150                     display_text_internal <= string_to_ascii("bEEr0150");
151                 when "100" =>
152                     selected_price <= to_unsigned(90, 14);
153                     display_text_internal <= string_to_ascii("h2o 0090");
154                 when others =>
155                     display_text_internal <= string_to_ascii("      ");
156             end case;
157

```

PART 3_PROCESS

```

158     when INSERT =>
159         price_str := format_price_to_string(selected_price);
160         inserted_str := format_price_to_string(inserted_amount);
161         full_str := price_str & inserted_str;
162         display_text_internal <= string_to_ascii(full_str);
163
164         -- -- --
165
166     when DISPENSE =>
167         if inserted_amount >= selected_price then
168             temp_change := inserted_amount - selected_price;
169         else
170             temp_change := (others => '0');
171         end if;
172
173         chg_str(1) := character'val(to_integer(temp_change / 1000 mod 10) + character'pos('0'));
174         chg_str(2) := character'val(to_integer(temp_change / 100 mod 10) + character'pos('0'));
175         chg_str(3) := character'val(to_integer(temp_change / 10 mod 10) + character'pos('0'));
176         chg_str(4) := character'val(to_integer(temp_change mod 10) + character'pos('0'));
177
178         display_text_internal <= string_to_ascii("chgE" & chg_str);
179
180     when others =>
181         display_text_internal <= string_to_ascii("      ");
182     end case;
183 end process;
184
185 end Behavioral;

```



VHDL CODE IMPLEMENTATIONS

COIN_HANDLER.VHD

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity Coin_Handler is
6  Port (
7      clk          : in  std_logic;
8      reset        : in  std_logic;
9      BTNU, BTNL, BTNR, BTND : in  std_logic;
10     inserted_amount : out unsigned(9 downto 0)
11 );
12 end Coin_Handler;
13
14 architecture Behavioral of Coin_Handler is
15
16     signal amount : unsigned(9 downto 0) := (others => '0');
17     signal prev_BTNU, prev_BTNL, prev_BTNR, prev_BTND : std_logic := '0';
18
19 begin
20
21     process(clk)
22     begin
23         if rising_edge(clk) then
24             if reset = '1' then
25                 amount <= (others => '0');
26             else
27                 if (BTNU = '1' and prev_BTNU = '0') then
28                     amount <= amount + 10;
29                 elsif (BTNL = '1' and prev_BTNL = '0') then
30                     amount <= amount + 20;
31                 elsif (BTNR = '1' and prev_BTNR = '0') then
32                     amount <= amount + 50;
33                 elsif (BTND = '1' and prev_BTND = '0') then
34                     amount <= amount + 100;
35                 end if;
36             end if;
37
38             prev_BTNU <= BTNU;
39             prev_BTNL <= BTNL;
40             prev_BTNR <= BTNR;
41             prev_BTND <= BTND;
42         end if;
43     end process;
44
45     inserted_amount <= amount;
46
47 end Behavioral;
48

```



VHDL CODE IMPLEMENTATIONS

DISPLAY_MANAGER.VHD

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity Display_Manager is
6  Port (
7      clk          : in  std_logic;
8      reset        : in  std_logic;
9      display_text : in  std_logic_vector(63 downto 0);
10     SEG          : out std_logic_vector(6 downto 0);
11     AN           : out std_logic_vector(7 downto 0);
12     dp           : out std_logic;
13 );
14 end Display_Manager;
15
16 architecture Behavioral of Display_Manager is
17
18     signal clk_divider : unsigned(17 downto 0) := (others => '0');
19     signal digit_index : integer range 0 to 7 := 0;
20     signal current_char : std_logic_vector(7 downto 0);
21     signal seg_temp : std_logic_vector(6 downto 0);
22
23 begin
24     process(clk)
25     begin
26         if rising_edge(clk) then
27             if reset = '1' then
28                 clk_divider <= (others => '0');
29                 digit_index <= 0;
30             else
31                 if clk_divider = 200000 - 1 then
32                     clk_divider <= (others => '0');
33                     if digit_index = 7 then
34                         digit_index <= 0;
35                     else
36                         digit_index <= digit_index + 1;
37                     end if;
38                 else
39                     clk_divider <= clk_divider + 1;
40                 end if;
41             end if;
42         end if;
43     end process;
44
45     current_char <= display_text(63 - digit_index*8 downto 56 - digit_index*8);
46

```

```

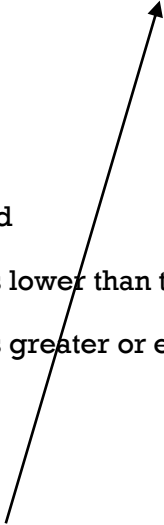
47     process(current_char)
48     begin
49         case current_char is
50             when x"20" => seg_temp <= "1111111"; -- space
51             when x"30" => seg_temp <= "0000001"; -- '0'
52             when x"31" => seg_temp <= "1001111"; -- '1'
53             when x"32" => seg_temp <= "0010010"; -- '2'
54             when x"33" => seg_temp <= "0000110"; -- '3'
55             when x"34" => seg_temp <= "1001100"; -- '4'
56             when x"35" => seg_temp <= "0100100"; -- '5'
57             when x"36" => seg_temp <= "0100000"; -- '6'
58             when x"37" => seg_temp <= "0001111"; -- '7'
59             when x"38" => seg_temp <= "0000000"; -- '8'
60             when x"39" => seg_temp <= "0000100"; -- '9'
61             when x"41" => seg_temp <= "0001000"; -- 'A'
62             when x"45" => seg_temp <= "0110000"; -- 'E'
63             when x"4F" => seg_temp <= "0000001"; -- '0'
64             when x"62" => seg_temp <= "1100000"; -- 'b'
65             when x"63" => seg_temp <= "1110010"; -- 'c'
66             when x"67" => seg_temp <= "0000100"; -- 'g'
67             when x"68" => seg_temp <= "1101000"; -- 'h'
68             when x"6C" => seg_temp <= "1001111"; -- 'l'
69             when x"6F" => seg_temp <= "1100010"; -- 'o'
70             when x"72" => seg_temp <= "1111000"; -- 'r'
71             when others => seg_temp <= "1111111"; -- blank
72         end case;
73     end process;
74
75     SEG <= seg_temp;
76     AN <= not std_logic_vector(to_unsigned(2**digit_index, 8));
77     dp <= '0' when (digit_index = 1 or digit_index = 5) else '1';
78
79 end Behavioral;

```



VERIFICATION WITH TEST BENCHES_RESULTS:FSM_CONTROLLER

One hot coding

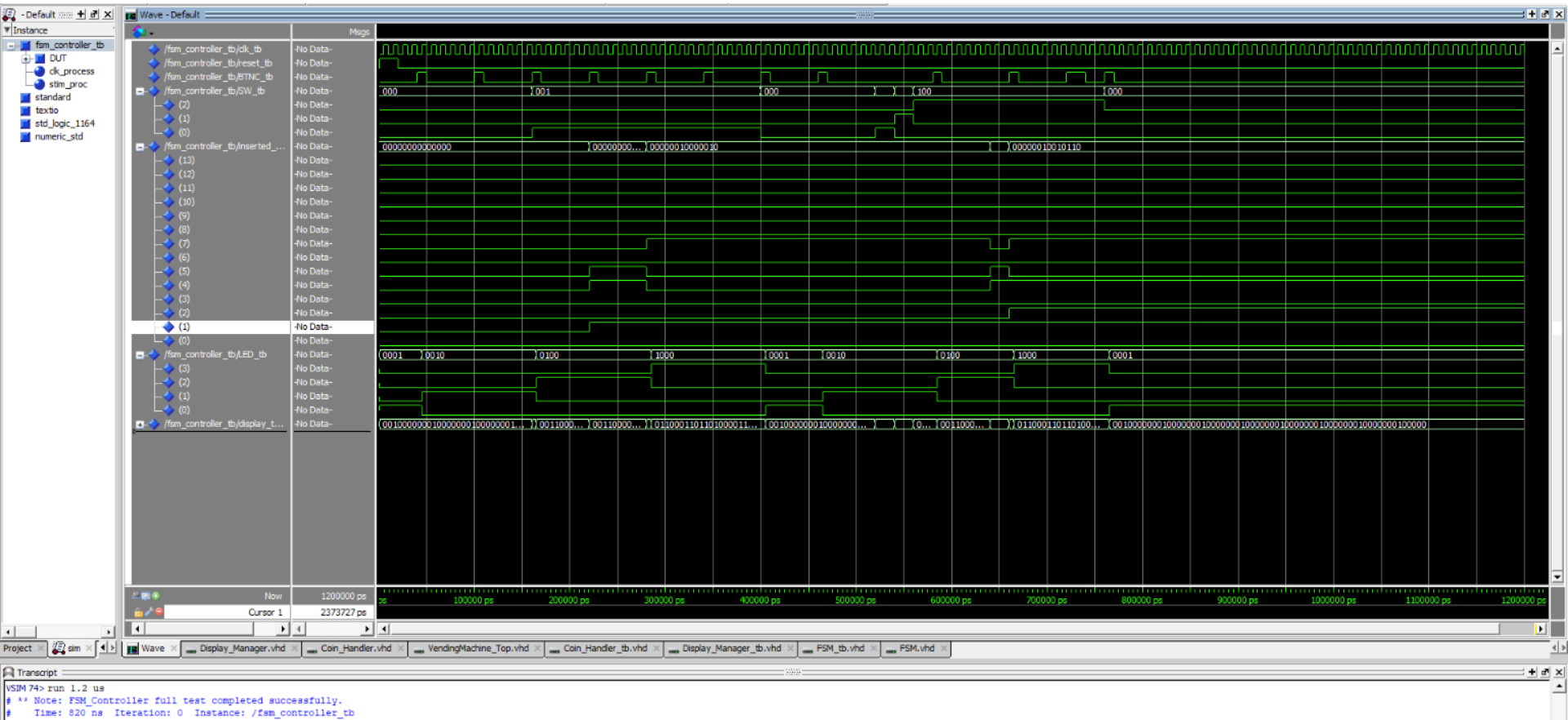


Test Cases:

-  Test case 1: Check if the state transitions from IDLE to DISPLAY when the BTNC is pressed
-  Test case 2: Check if the state transitions from DISPLAY to INSERT when no switch is selected
-  Test case 3: Check if the state transitions from DISPLAY to INSERT when switch 1 is selected
-  Test case 4: Check if the state transitions from INSERT to DISPENSE when inserted amount is lower than the price of the item
-  Test case 5: Check if the state transitions from INSERT to DISPENSE when inserted amount is greater or equal to the price of the item
-  Test case 6: Check if the state transitions from DISPENSE to IDLE when a switch is still on
-  Test case 7: Check if the state transitions from DISPENSE to IDLE when a switch is finally off.
-  Test case 8: Check if the state transitions from IDLE to DISPLAY when the BTNC is pressed.
-  Test case 9: Check if the item 1 is displayed correctly if the SW(0) is on. (SW_tb = "001")
-  Test case 10: Check if the item 2 is displayed correctly if the SW(1) is on. (SW_tb = "010")
-  Test case 11: Check if the item 3 is displayed correctly if the SW(2) is on. (SW_tb = "100")
-  Test case 12: Check if the inserted amount (50) is correctly being displayed on the right 4 digits of 7 segment display.
-  Test case 13: Check if the inserted amount (150) allows state transition to DISPENSE and the correct change is displayed on the 7 segment display.
-  Test case 14: Check if the state transitions from DISPENSE to IDLE when a switch is still on.
-  Test case 15: Check if the state transitions from DISPENSE to IDLE when the switch is finally off.



TEST BENCHES_RESULTS:FSM_CONTROLLER



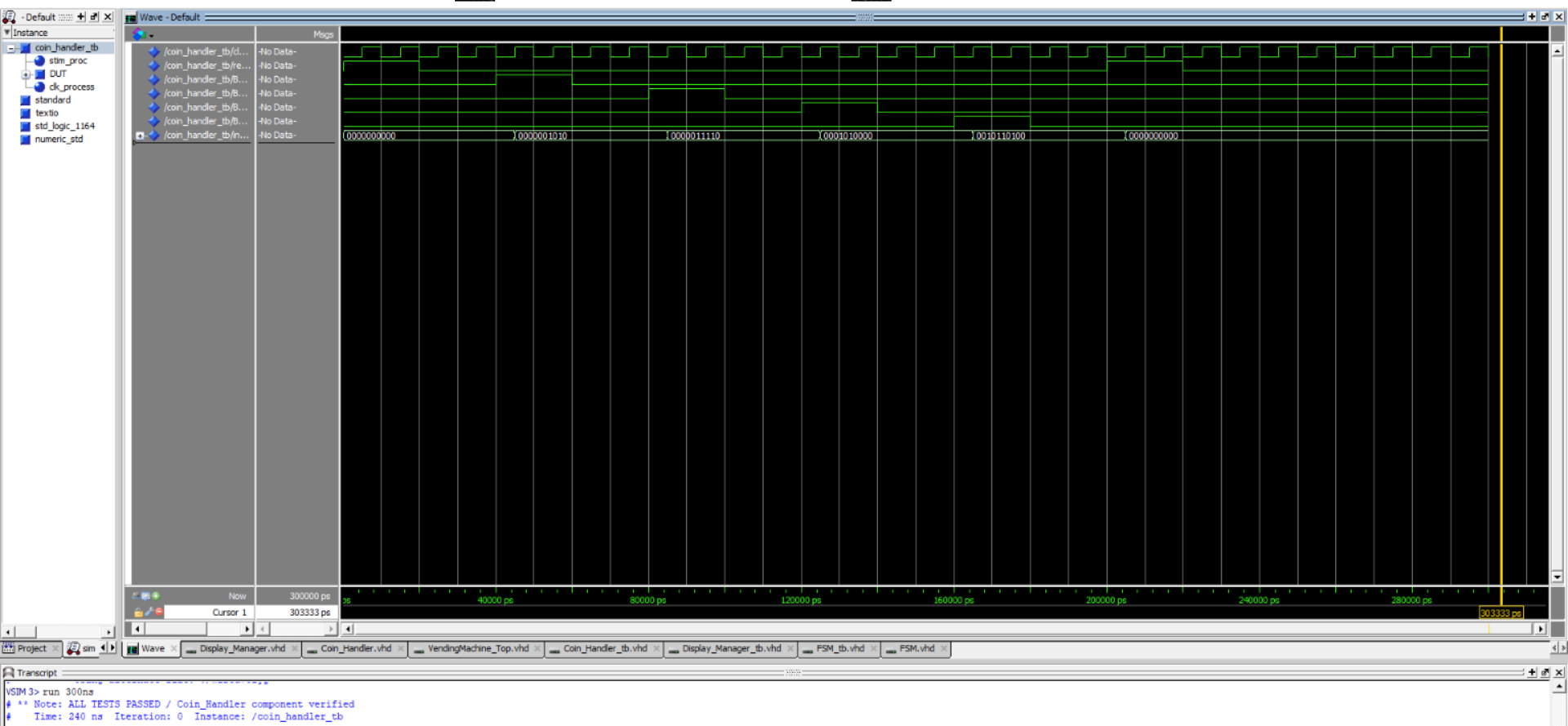
VERIFICATION WITH TEST BENCHES_RESULTS:COIN_HANDLER

Test Cases:

- ☞ '01234567'
- ☞ '89 AEObc'
- ☞ 'ghlor '
- ☞ 'colA0130'
- ☞ 'bEEr0150'
- ☞ 'h2o 0090'



TEST BENCHES_RESULTS:COIN_HANDLER



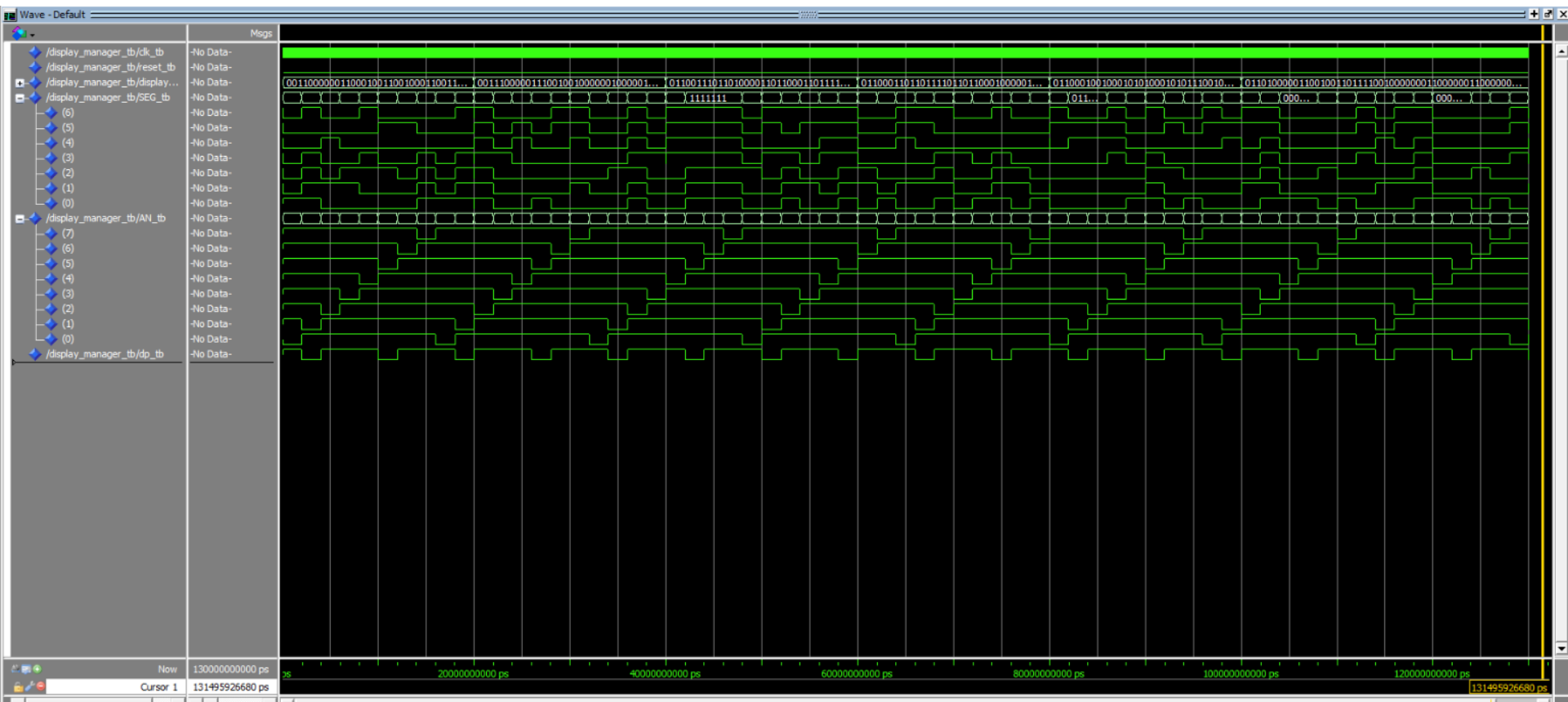
VERIFICATION WITH TEST BENCHES_RESULTS:DISPLAY_MANAGER

Test Cases:

-  Insert 10 cents and see if the inserted amount is 10 cents
-  Insert 20 cents and see if the inserted amount is 30 cents
-  Insert 50 cents and see if the inserted amount is 80 cents
-  Insert 100 cents = 1 euro and see if the inserted amount is 180 cents
-  Reset the test



TEST_BENCHES_RESULTS:DISPLAY_MANAGER



FPGA IMPLEMENTATIONS CONSTRAINT FILE

```

1  # Clock input (used for digit switching)
2  set_property -dict { PACKAGE_PIN E3 IOSTANDARD LVCMOS33 } [get_ports { clk }];
3  create_clock -add -name sys_clk_pin -period 20.00 -waveform {0 10} [get_ports {clk}];
4  # reset
5  set_property PACKAGE_PIN V10 [get_ports {reset}]
6  set_property IOSTANDARD LVCMOS33 [get_ports {reset}]
7  # buttons
8  set_property PACKAGE_PIN N17 [get_ports {BTNC}]
9  set_property IOSTANDARD LVCMOS33 [get_ports {BTNC}]
10 set_property PACKAGE_PIN M18 [get_ports {BTNU}]
11 set_property IOSTANDARD LVCMOS33 [get_ports {BTNU}]
12 set_property PACKAGE_PIN P17 [get_ports {BTNL}]
13 set_property IOSTANDARD LVCMOS33 [get_ports {BTNL}]
14 set_property PACKAGE_PIN M17 [get_ports {BTNR}]
15 set_property IOSTANDARD LVCMOS33 [get_ports {BTNR}]
16 set_property PACKAGE_PIN P18 [get_ports {BTND}]
17 set_property IOSTANDARD LVCMOS33 [get_ports {BTND}]
18 # sw
19 set_property PACKAGE_PIN J15 [get_ports {SW[0]}]
20 set_property IOSTANDARD LVCMOS33 [get_ports {SW[0]}]
21 set_property PACKAGE_PIN L16 [get_ports {SW[1]}]
22 set_property IOSTANDARD LVCMOS33 [get_ports {SW[1]}]
23 set_property PACKAGE_PIN M13 [get_ports {SW[2]}]
24 set_property IOSTANDARD LVCMOS33 [get_ports {SW[2]}]
25 # LED
26 set_property PACKAGE_PIN V11 [get_ports {LED[0]}]
27 set_property IOSTANDARD LVCMOS33 [get_ports {LED[0]}]
28 set_property PACKAGE_PIN V12 [get_ports {LED[1]}]
29 set_property IOSTANDARD LVCMOS33 [get_ports {LED[1]}]
30 set_property PACKAGE_PIN V14 [get_ports {LED[2]}]
31 set_property IOSTANDARD LVCMOS33 [get_ports {LED[2]}]
32 set_property PACKAGE_PIN V15 [get_ports {LED[3]}]
33 set_property IOSTANDARD LVCMOS33 [get_ports {LED[3]}]
34 # 7 SEG[6:0]
35 # 7 段数码管段选 SEG[6:0]
36 set_property PACKAGE_PIN L18 [get_ports {SEG[0]}]
37 set_property IOSTANDARD LVCMOS33 [get_ports {SEG[0]}]
38 set_property PACKAGE_PIN T11 [get_ports {SEG[1]}]
39 set_property IOSTANDARD LVCMOS33 [get_ports {SEG[1]}]
40 set_property PACKAGE_PIN P15 [get_ports {SEG[2]}]
41 set_property IOSTANDARD LVCMOS33 [get_ports {SEG[2]}]
42 set_property PACKAGE_PIN K13 [get_ports {SEG[3]}]
43 set_property IOSTANDARD LVCMOS33 [get_ports {SEG[3]}]
44 set_property PACKAGE_PIN K16 [get_ports {SEG[4]}]
45 set_property IOSTANDARD LVCMOS33 [get_ports {SEG[4]}]
46 set_property PACKAGE_PIN R10 [get_ports {SEG[5]}]
47 set_property IOSTANDARD LVCMOS33 [get_ports {SEG[5]}]
48 set_property PACKAGE_PIN T10 [get_ports {SEG[6]}]
49 set_property IOSTANDARD LVCMOS33 [get_ports {SEG[6]}]
50 # Anode control
51 set_property -dict { PACKAGE_PIN J17 IOSTANDARD LVCMOS33 } [get_ports { AN[7] }];
52 set_property -dict { PACKAGE_PIN J18 IOSTANDARD LVCMOS33 } [get_ports { AN[6] }];
53 set_property -dict { PACKAGE_PIN T9 IOSTANDARD LVCMOS33 } [get_ports { AN[5] }];
54 set_property -dict { PACKAGE_PIN J14 IOSTANDARD LVCMOS33 } [get_ports { AN[4] }];
55 set_property -dict { PACKAGE_PIN P14 IOSTANDARD LVCMOS33 } [get_ports { AN[3] }];
56 set_property -dict { PACKAGE_PIN T14 IOSTANDARD LVCMOS33 } [get_ports { AN[2] }];
57 set_property -dict { PACKAGE_PIN K2 IOSTANDARD LVCMOS33 } [get_ports { AN[1] }];
58 set_property -dict { PACKAGE_PIN U13 IOSTANDARD LVCMOS33 } [get_ports { AN[0] }];
59 # dp
60 set_property PACKAGE_PIN H15 [get_ports {dp}]
61 set_property IOSTANDARD LVCMOS33 [get_ports {dp}]

```



FPGA SIMULATIONS

Videos



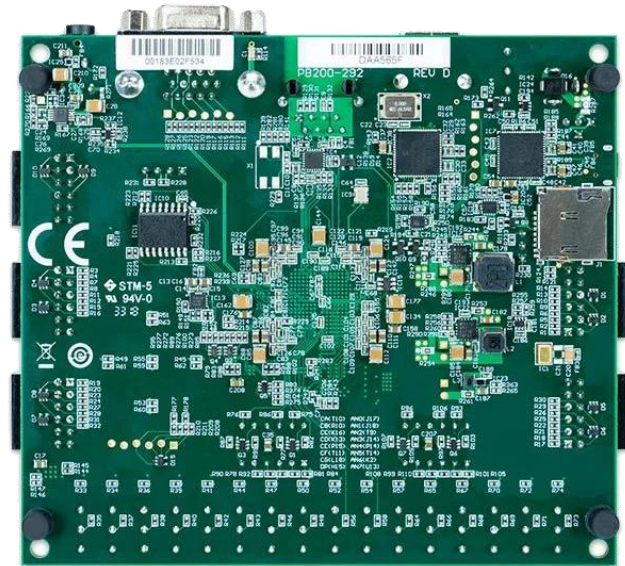
Simulation in Real Time



PCB PROTOTYPING

PCB Prototyping is a process of designing printed circuit boards in which we electrically connect electronic components. Some known process of PCB Prototype

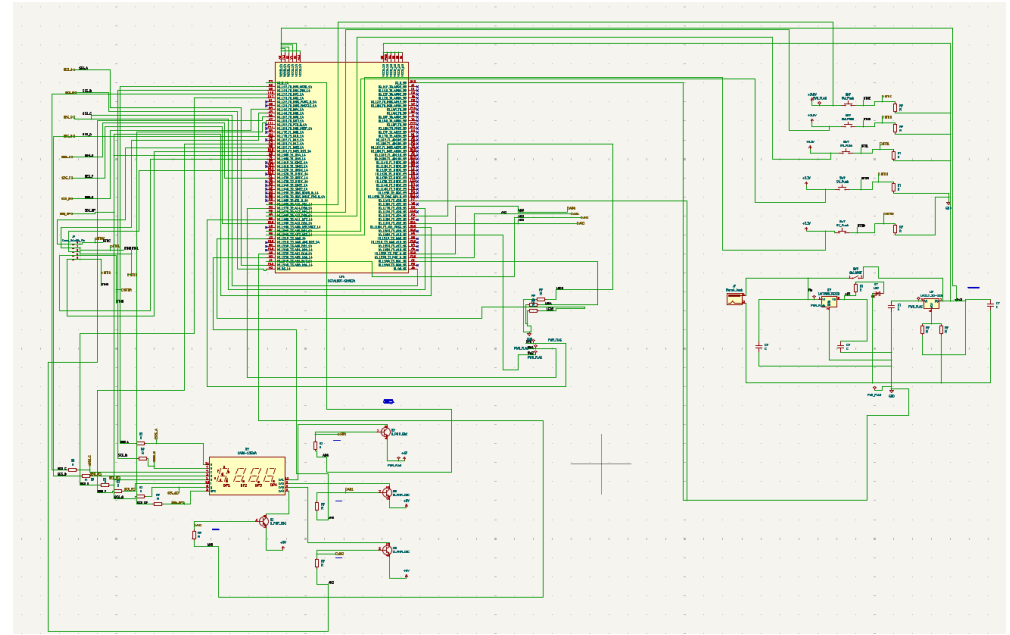
1. Find requirement according the target group or client
2. Select the design tools
3. Schematics drawing
4. Components placements
5. Routing
6. Design Rules Check
7. Gerber file generation
8. Simulation and testing
9. Prototyping
10. Final production



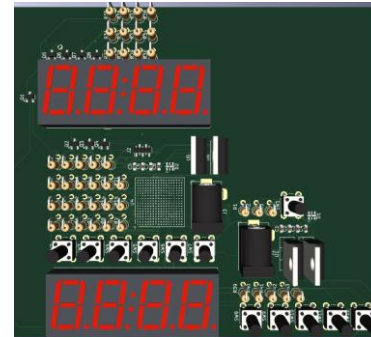
PCB PROTOTYPING

After discussion and deep analysis we select these elements for our PCB

- ❏ Two 7 segment display
- ❏ Power supply
- ❏ Resistors and Capacitors
- ❏ Central component is the Nexus A7 *XC7A100T* FPGA, handling all control logic and I/O
- ❏ Connected to multiple push buttons for item selection and user interaction.
- ❏ **PNP and NPN transistors control outputs like relays or motors, acting as switches via FPGA signals.**



Schematics



3D View



PCB PROTOTYPING

Our custom PCB Board Design Process

- ❌ I did follow some steps to creating our custom PCB
- ❌ Initially I did install Kicad 9.0 for designing
- ❌ I did create a folder for save our custom board.
- ❌ I start add components from library to schematics
- ❌ After complete schematics then I start routing
- ❌ I added external plugin for auto routing like freerouting
- ❌ I added some connection manually
- ❌ I am able to see the nice 3D view of custom PCB
- ❌ Now I am able to generate Gerber file

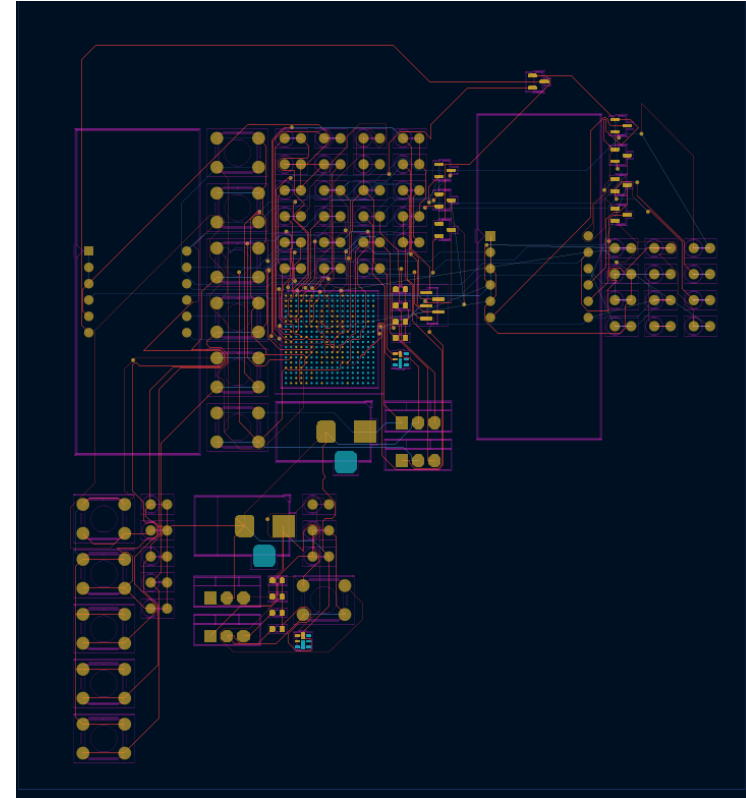


Fig: PCB Layout



BOM COST AND GERBER FILES

BOM stands for Bill of Material that:

- Is a detailed list of all components used in the PCB.
- Includes part numbers, quantities, footprints, and specifications.
- Helps manufacturers know what parts to purchase and assemble.
- Generated automatically by the PCB design software (e.g., KiCad).

Gerber files are:

- set of files that describe physical layout of PCB.
- Includes copper layers, silkscreen, solder mask, and drill holes.
- Used by PCB manufacturers to **fabricate the board**.

Board stats	Front	Back	Total
Components	83	0	83
Groups	12	0	12
SMD pads	385	0	385
TH pads	176		

- 1 vending_machine_pcb-B_Cu.gbr (Copper, L2)
- 2 vending_machine_pcb-B_Paste.gbr (Paste, Bot)
- 3 vending_machine_pcb-Edge_Cuts.gbr (Profile, NP)
- 4 vending_machine_pcb-F_Mask.gbr (Soldermask, Top)
- 5 vending_machine_pcb-F_Silkscreen.gbr (Legend, Top)
- 6 vending_machine_pcb-B_Mask.gbr (Soldermask, Bot)
- 7 vending_machine_pcb-B_Silkscreen.gbr (Legend, Bot)
- 8 vending_machine_pcb-F_Cu.gbr (Copper, L1)
- 9 vending_machine_pcb-F_Paste.gbr (Paste, Top)

	Sourced	Placed	References	Value	Footprint	Quantity
1	☐	☐	C1, C2, C3, C4, C5, C6, C7, C8	C	C_0603_1608Metric	8
2	☐	☐	R1, R2, R3, R4, R5, R6, R7, R8, R9, R10, R11, R12, R13, R14, R15, R16, R17, R18, R19, R20, R21, R22, R23, R24, R25, R26, R27, R28, R29, R30, R31, R32, R33, R34, R35, R36, R37, R38, R39, R40, R41, R42, R43	R	R_Axial_DIN0207_L6.3mm_D2.5mm_P2.54mm_Veritical	43
3	☐	☐	D1, D2	LED	LED-APA102-2020	2
4	☐	☐	U1, U5	LH7805_T0220	TO-220-3_Veritical	2
5	☐	☐	U2, U6	LM317_TO-220	TO-220-3_Veritical	2
6	☐	☐	U3, U7	CA56-12ENA	CA56-12ENA	2
7	☐	☐	U4	XC7A100T-CSG324	BGA-324_15x15mm_Layout18x18_P0.8mm	1
8	☐	☐	SW2, SW3, SW4, SW5, SW6, SW8, SW9, SW10, SW11, SW12	SM_Push	SM_PUSH_6mm_H13mm	10
9	☐	☐	SW1, SW7	SM_SPST	SM_PUSH_6mm_H13mm	2
10	☐	☐	Q1, Q2, Q3, Q4, Q5, Q6, Q7, Q8	Q_PNP_EBC	SOT-23	8
11	☐	☐	J1, J3	Barrel_Jack	BarrelJack_Horizontal	2
12	☐	☐	J2	Conn_01x05_Pin	PinSocket_1x05_P1.00mm_Veritical_SMD_Pin1left	1

Charge Details

Engineering fee

\$4.00

Via Covering

\$0.00

Large Size

\$163.30

Surface Finish

\$0.00

Board

\$87.00

Build Time

PCB: 2 days \$0.00

24 hours \$7.20

24 hours

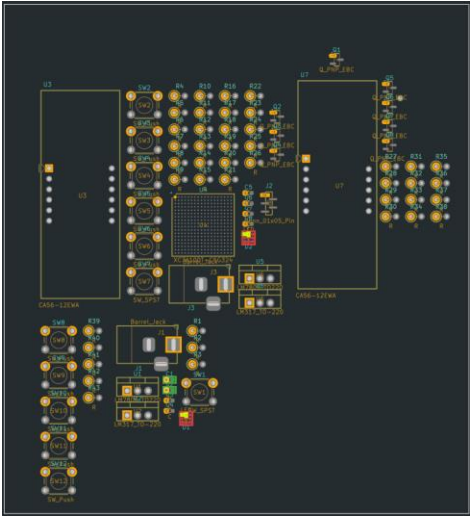
PCBA Only

 \$0.00

Calculated Price

\$254.30

Additional charges may apply for special cases



Thank you for listening

**We are ready to show simulation on FPGA and
answer your questions**

